Final Project

Lily Monte

2025-04-24

```
rm(list = ls())

library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.1.4     v readr     2.1.5
## v forcats   1.0.0     v stringr   1.5.1
## v ggplot2   3.5.1     v tibble    3.2.1
## v lubridate 1.9.4     v tidyr     1.3.1
## v purrr     1.0.2
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(tidycensus)
```

```
## Warning: package 'tidycensus' was built under R version 4.4.3
```

```
library(sf)
```

```
## Linking to GEOS 3.12.2, GDAL 3.9.3, PROJ 9.4.1; sf_use_s2() is TRUE
```

```
library(ggspatial)
library(osmdata)
```

```
## Warning: package 'osmdata' was built under R version 4.4.3

## Data (c) OpenStreetMap contributors, ODbL 1.0. https://www.openstreetmap.org/copyright
```

```
library(prettymapr)
```

```
## Warning: package 'prettymapr' was built under R version 4.4.3
```

```
library(leaflet)
library(nabor)
```

```
## Warning: package 'nabor' was built under R version 4.4.3
```

```r
library(stargazer)
```

```
##
## Please cite as:
##
##  Hlavac, Marek (2022). stargazer: Well-Formatted Regression and Summary Statistics Tables.
##  R package version 5.2.3. https://CRAN.R-project.org/package=stargazer
```

```r
library(units)
```

```
## udunits database from C:/Program Files/R/R-4.4.2/library/units/share/udunits/udunits2.xml
```

```r
setwd("C:/Users/Lily/Documents/collegefiles/PLAN 372/final")

#buildings = st_read("north-carolina-latest-free.shp/gis_osm_buildings_a_free_1.shp")
placesarea = st_read("north-carolina-latest-free.shp/gis_osm_places_a_free_1.shp")
```

```
## Reading layer ‘gis_osm_places_a_free_1’ from data source
##   ‘C:\Users\Lily\Documents\collegefiles\PLAN 372\final\north-carolina-latest-free.shp\gis_osm_places
##   using driver ‘ESRI Shapefile’
## Simple feature collection with 962 features and 5 fields
## Geometry type: MULTIPOLYGON
## Dimension:     XY
## Bounding box:  xmin: -84.32183 ymin: 33.75288 xmax: -75.40012 ymax: 36.58816
## Geodetic CRS:  WGS 84
```

```r
placespoints = st_read("north-carolina-latest-free.shp/gis_osm_places_free_1.shp")
```

```
## Reading layer ‘gis_osm_places_free_1’ from data source
##   ‘C:\Users\Lily\Documents\collegefiles\PLAN 372\final\north-carolina-latest-free.shp\gis_osm_places
##   using driver ‘ESRI Shapefile’
## Simple feature collection with 6253 features and 5 fields
## Geometry type: POINT
## Dimension:     XY
## Bounding box:  xmin: -84.30436 ymin: 33.85712 xmax: -75.46781 ymax: 36.57373
## Geodetic CRS:  WGS 84
```

```r
blockgroups = st_read("tl_2020_37_bg.shp")
```

```
## Reading layer ‘tl_2020_37_bg’ from data source
##   ‘C:\Users\Lily\Documents\collegefiles\PLAN 372\final\tl_2020_37_bg.shp’
##   using driver ‘ESRI Shapefile’
## Simple feature collection with 7111 features and 12 fields
## Geometry type: MULTIPOLYGON
## Dimension:     XY
## Bounding box:  xmin: -84.32182 ymin: 33.75288 xmax: -75.40012 ymax: 36.58814
## Geodetic CRS:  NAD83
```

```
vehicles = st_read("SimplyAnalytics_Shapefiles_b50ce4268cd95a605b2cde956126e92327c8a29c674a636b8a93c819
```

```
## Reading layer 'SimplyAnalytics_Shapefiles_b50ce4268cd95a605b2cde956126e92327c8a29c674a636b8a93c819dea
##   using driver 'ESRI Shapefile'
## Simple feature collection with 597 features and 3 fields
## Geometry type: POLYGON
## Dimension:     XY
## Bounding box:  xmin: -78.99505 ymin: 35.51946 xmax: -78.25371 ymax: 36.07644
## Geodetic CRS:  WGS 84
```

```
blockgroups = blockgroups %>%
  filter(COUNTYFP == "183")
```

```
st_crs(blockgroups)
```

```
## Coordinate Reference System:
##   User input: NAD83
##   wkt:
## GEOGCRS["NAD83",
##     DATUM["North American Datum 1983",
##         ELLIPSOID["GRS 1980",6378137,298.257222101,
##             LENGTHUNIT["metre",1]]],
##     PRIMEM["Greenwich",0,
##         ANGLEUNIT["degree",0.0174532925199433]],
##     CS[ellipsoidal,2],
##         AXIS["latitude",north,
##             ORDER[1],
##             ANGLEUNIT["degree",0.0174532925199433]],
##         AXIS["longitude",east,
##             ORDER[2],
##             ANGLEUNIT["degree",0.0174532925199433]],
##     ID["EPSG",4269]]
```

```
blockgroups = st_transform(blockgroups, 4326)
vehicles = st_transform(vehicles, 4326)
#ensures all data uses the same EPSG Code

raleigh = filter(placesarea, name == "Raleigh" | name == "Cary" | name == "Garner" | name == "Apex" | na

lawoffices = raleigh %>%
  st_bbox() %>%
  opq() %>%
  #add_osm_feature(key = "office", value = "lawyer") %>%
  add_osm_features (features = c (
        "\"office\"=\"lawyer\"",
        "\"amenity\"=\"courthouse\"",
        "\"office\"=\"notary\"",
        "\"government\"=\"administrative\"",
        "\"government\"=\"register_office\"",
        "\"government\"=\"prosecutor\"")) %>%
  #add_osm_feature(key = "amenity", value = "courthouse") %>%
  osmdata_sf()
```

```
#The above code pulls legal services from the osm database

#highways = raleigh %>%
  #st_bbox() %>%
  #opq() %>%
  #add_osm_features(features = list(
   # "highway" = "motorway",
   # "highway" = "primary",
   # "highway" = "secondary",
   # "highway" = "tertiary"
 # )) %>%
  #osmdata_sf()

lawoffices
```

```
## Object of class 'osmdata' with:
##                  $bbox : 35.5194581,-78.9950674,36.0765416,-78.254531
##         $overpass_call : The call submitted to the overpass API
##                  $meta : metadata including timestamp and version numbers
##            $osm_points : 'sf' Simple Features Collection with 254 points
##             $osm_lines : NULL
##          $osm_polygons : 'sf' Simple Features Collection with 24 polygons
##        $osm_multilines : NULL
##     $osm_multipolygons : NULL
```

```
lawofficespoints = bind_rows(
  pluck(lawoffices, "osm_points"),
  st_centroid(pluck(lawoffices, "osm_polygons"))
)
```

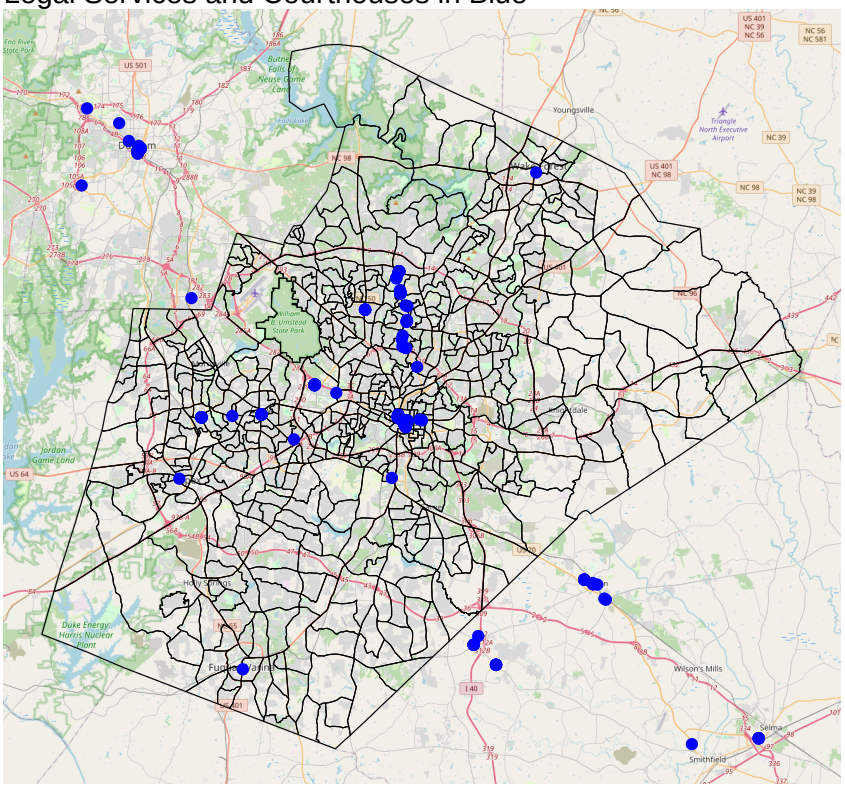
```
## Warning: st_centroid assumes attributes are constant over geometries
```

```
#converts the obtained spatial data into a spataial dataframe

#highwaylines = pluck(highways, "osm_lines")


ggplot()+
  annotation_map_tile(type = "osm", zoomin = 0) +
  geom_sf(data = blockgroups, color = "black", fill = NA)+
  #geom_sf(data = highwaylines, color = "black") +
  geom_sf(data = lawofficespoints, color = "blue") +
  theme_void() +
  labs(title = "Fig. 1. Map of Wake County",
       subtitle = "Legal Services and Courthouses in Blue")
```

```
## Zoom: 11
```

Fig. 1. Map of Wake County

Legal Services and Courthouses in Blue



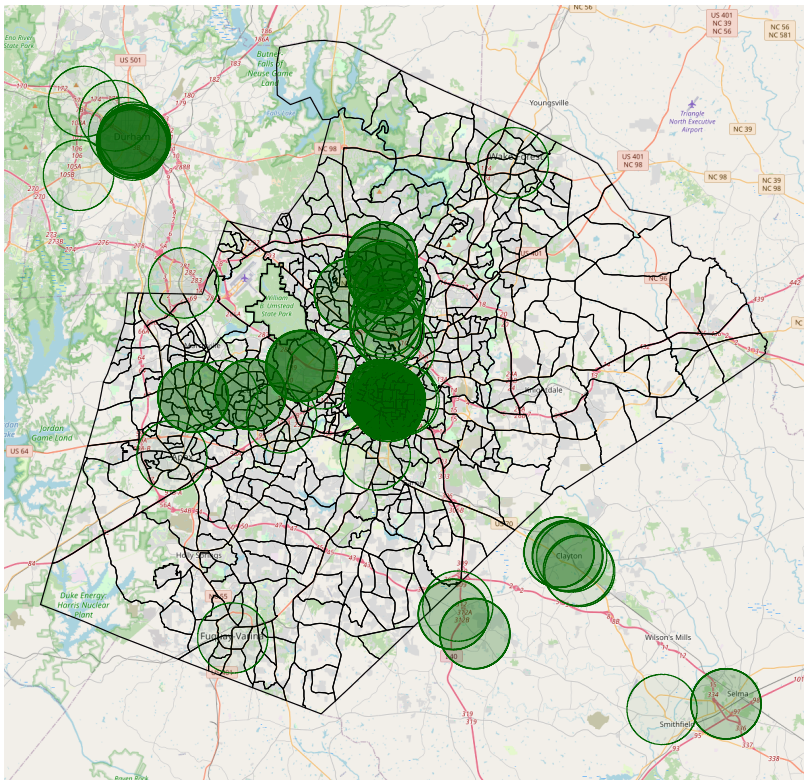

```
#plots blockgroups and legal services.
```

```
buffer = lawofficespoints %>%
  st_buffer(dist = 3218.69)

access = ggplot() +
    annotation_map_tile(type = "osm", zoomin = 0) +
    geom_sf(data = blockgroups, color = "black", fill = NA) +
    geom_sf(data = buffer, alpha = 0.05, fill = "darkgreen", color = "darkgreen")+
    theme_void() +
    labs(title = "Fig. 2. Map of Wake County",
       subtitle = "Accessible Areas in Green")
access
```

```
## Zoom: 11
```

Fig. 2. Map of Wake County

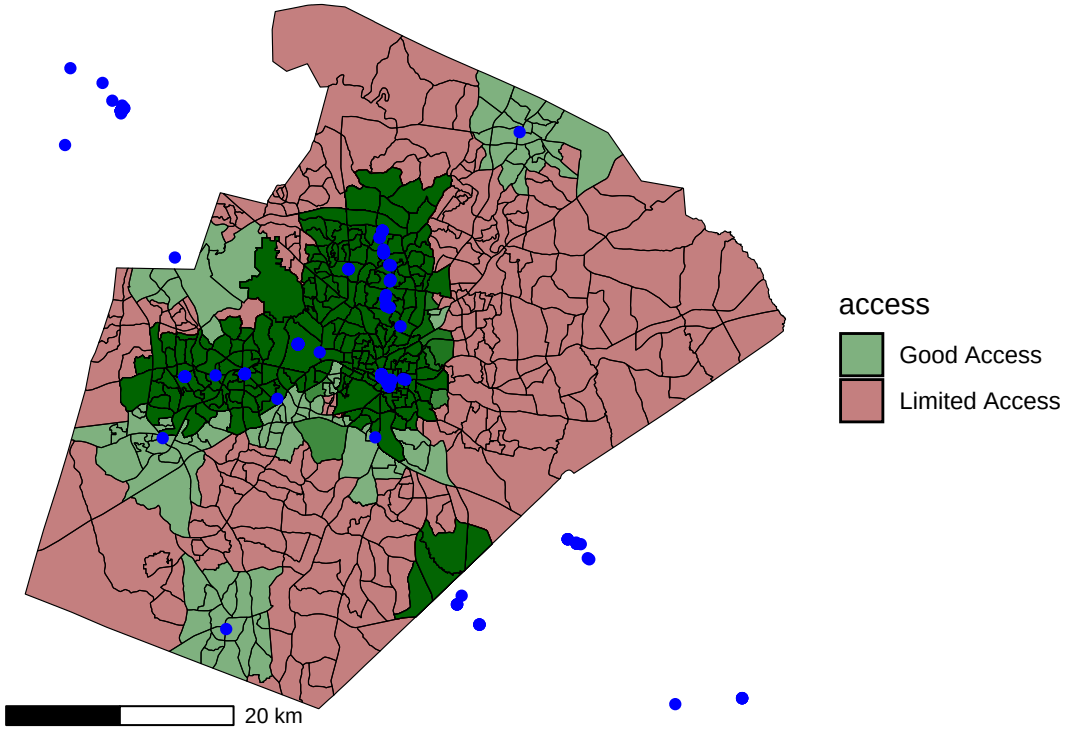
Accessible Areas in Green




```
join = blockgroups %>%
  st_join(buffer,
          join = st_intersects,
          left = T) %>%
  mutate(access = ifelse(is.na(osm_id),"Limited Access", "Good Access")) %>%
  mutate(access = as.factor(access))

Access2 = ggplot() +
  scale_fill_manual(values = c("darkgreen","darkred")) +
  geom_sf(data = join, color = "black",  alpha = 0.5, aes(fill = access))+
  geom_sf(data = lawofficespoints, color = "blue") +
  theme_void() +
  labs(title="Fig. 3. Access to Legal Services in Wake County",
       subtitle = "By Census Block Group") +
  annotation_scale(location = "bl", width_hint = 0.3)

Access2
```

Fig. 3. Access to Legal Services in Wake County

By Census Block Group




```
v24 <- load_variables(2023, "acs5", cache = TRUE)
v24 = v24 %>%
  filter(geography == "block group")

blockgroupcensus = get_acs(geography = "cbg",
        variables = "B19013_001",
        year = 2023,
        state = "NC",
        county = "Wake")
```

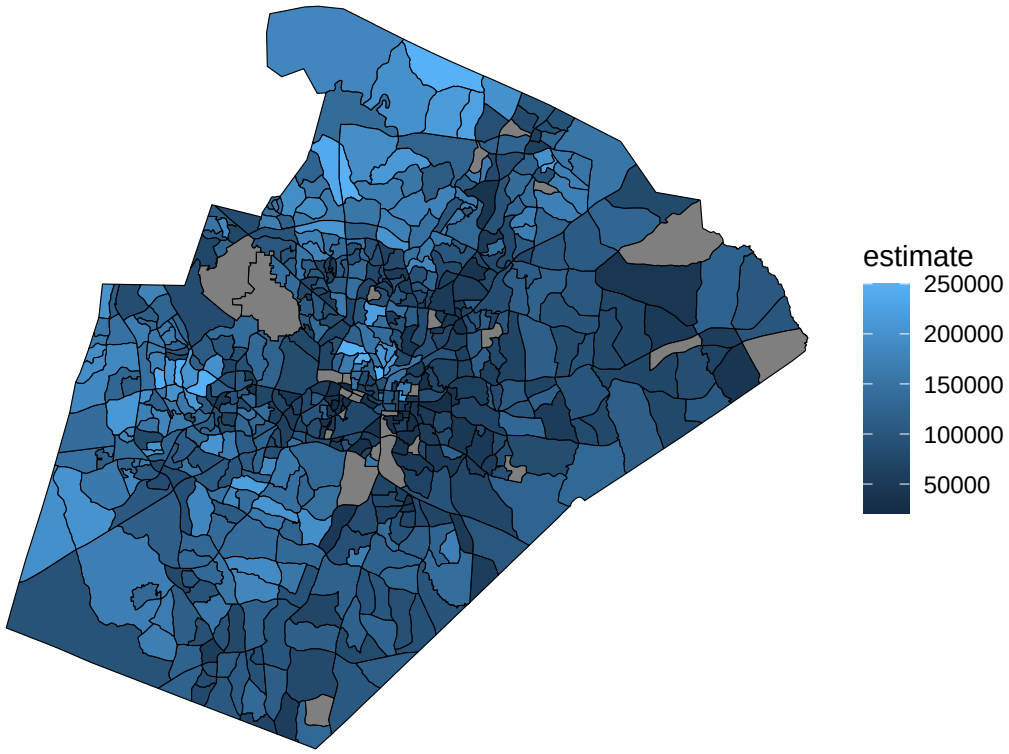
```
## Getting data from the 2019-2023 5-year ACS
```

```
censusblocks = left_join(blockgroups, blockgroupcensus)
```

```
## Joining with `by = join_by(GEOID)`
```

```
income = ggplot(data = censusblocks, aes(fill = estimate)) +
    #annotation_map_tile(type = "osm", zoomin = 0) +
    geom_sf(color = "black") +
    theme_void() +
    labs(title = "Fig. 4. Map of Wake County Census Tracts by Income")
income
```

Fig. 4. Map of Wake County Census Tracts by Income



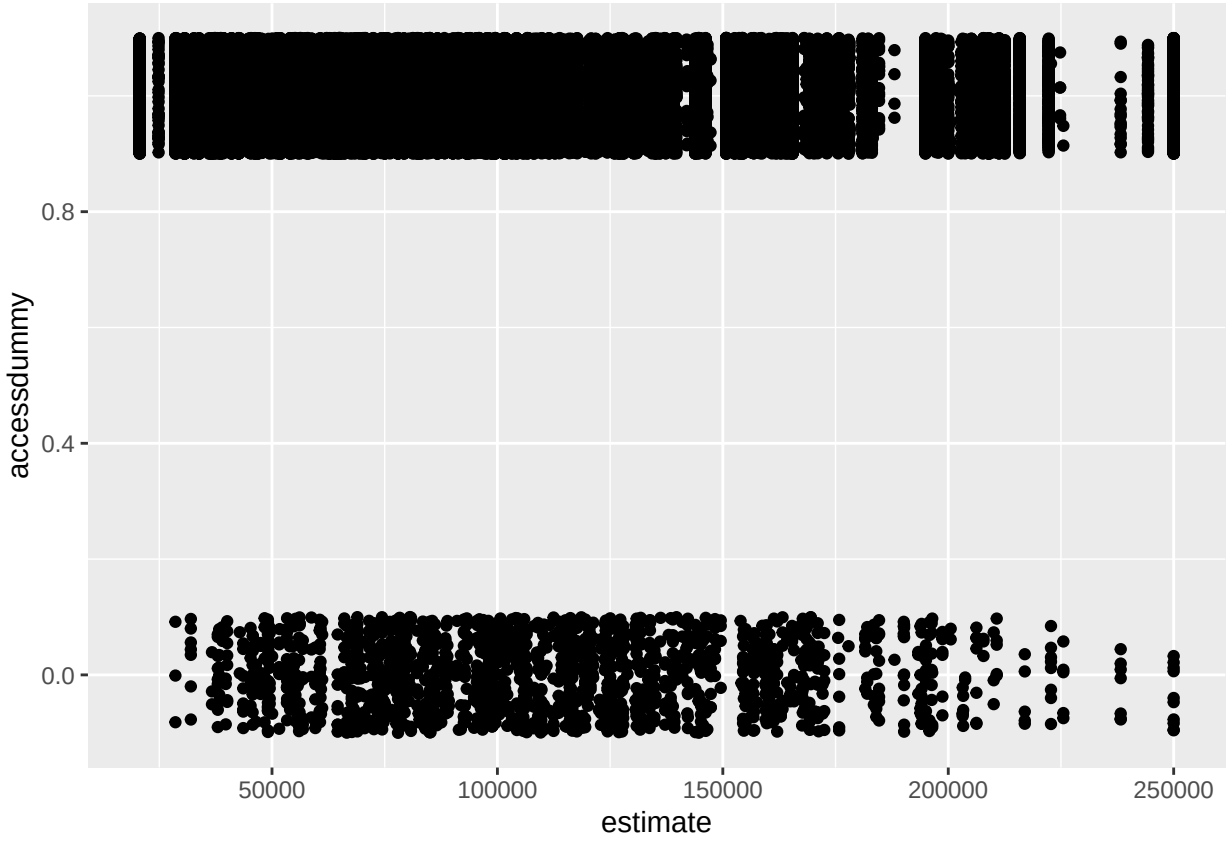

```
accessjoin = join %>%
  select(access, GEOID)
accessjoin = st_join(censusblocks, accessjoin)

accessjoin = accessjoin %>%
  mutate(accessdummy = ifelse(access == "Good Access", 1, 0))

fit1 = lm(accessdummy ~ estimate, data = accessjoin)
summary(fit1)
```

```
##
## Call:
## lm(formula = accessdummy ~ estimate, data = accessjoin)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.98432  0.01935  0.02335  0.03041  0.04765
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)  9.884e-01  1.253e-03  788.58   <2e-16 ***
## estimate    -1.444e-07  1.126e-08  -12.82   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
```

```
## Residual standard error: 0.1578 on 67181 degrees of freedom
##   (7061 observations deleted due to missingness)
## Multiple R-squared:  0.002441,   Adjusted R-squared:  0.002426
## F-statistic: 164.4 on 1 and 67181 DF,  p-value: < 2.2e-16
```

```
plot1 = ggplot()+
  geom_jitter(data = accessjoin, aes(x = estimate, y = accessdummy), height = 0.1)
plot1
```

```
## Warning: Removed 7061 rows containing missing values or values outside the scale range
## (`geom_point()`).
```




```
stargazer(fit1)
```

```
##
## % Table created by stargazer v.5.2.3 by Marek Hlavac, Social Policy Institute. E-mail: marek.hlavac a
## % Date and time: Thu, Apr 24, 2025 - 10:30:25
## \begin{table}[!htbp] \centering
##   \caption{}
##   \label{}
## \begin{tabular}{@{\extracolsep{5pt}}lc}
## \\[-1.8ex]\hline
## \hline \\[-1.8ex]
##  & \multicolumn{1}{c}{\textit{Dependent variable:}} \\
```

```
## \cline{2-2}
## \\[-1.8ex] & accessdummy \\
## \hline \\[-1.8ex]
##  estimate & $-$0.00000$^{***}$ \\
##   & (0.000) \\
##   & \\
##  Constant & 0.988$^{***}$ \\
##   & (0.001) \\
##   & \\
## \hline \\[-1.8ex]
## Observations & 67,183 \\
## R$^{2}$ & 0.002 \\
## Adjusted R$^{2}$ & 0.002 \\
## Residual Std. Error & 0.158 (df = 67181) \\
## F Statistic & 164.389$^{***}$ (df = 1; 67181) \\
## \hline
## \hline \\[-1.8ex]
## \textit{Note:}  & \multicolumn{1}{r}{$^{*}$p$<$0.1; $^{**}$p$<$0.05; $^{***}$p$<$0.01} \\
## \end{tabular}
## \end{table}
```

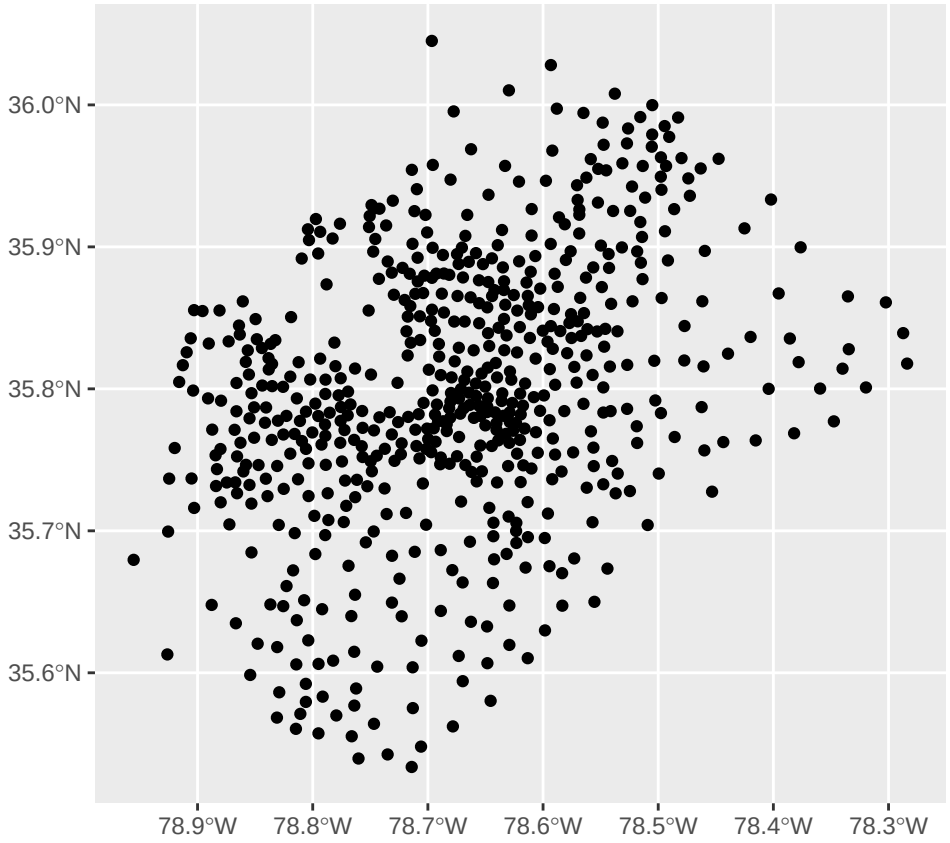
```
vehiclesjoin = st_join(accessjoin, vehicles)

fit2 = lm(accessdummy ~ VALUE0, data = vehiclesjoin)
summary(fit2)
```

```
##
## Call:
## lm(formula = accessdummy ~ VALUE0, data = vehiclesjoin)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.00534  0.02010  0.02554  0.02907  0.02907
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) 9.709e-01  2.669e-04 3637.93   <2e-16 ***
## VALUE0      1.360e-04  3.995e-06   34.05   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.1507 on 533169 degrees of freedom
## Multiple R-squared:  0.002169,   Adjusted R-squared:  0.002167
## F-statistic:  1159 on 1 and 533169 DF,  p-value: < 2.2e-16
```

```
blockgroupcentroids = st_centroid(blockgroups)
```

```
## Warning: st_centroid assumes attributes are constant over geometries
```

```
#calculating the centroid of each block group

ggplot() +
  geom_sf(data = blockgroupcentroids, color = "black")
```




```
blockgroupcentroids$type = "blockgroup"
lawofficespoints$type = "lawoffice"

blockgroupcentroids = blockgroupcentroids %>%
  mutate(nodeID = c(1:n()))

lawofficespoints = lawofficespoints %>%
  mutate(nodeID = c(1:n()))

nearestindices = st_nearest_feature(blockgroupcentroids, lawofficespoints)

nearest_points = lawofficespoints[nearestindices, ]

nearest_distances <- st_distance(blockgroupcentroids, nearest_points, by_element = TRUE)
nearest_distances_mi = set_units(nearest_distances, "mi")

distanceresults = cbind(
  #st_drop_geometry(blockgroupcentroids), # Drop the geometry to avoid duplication
  nearest_point_id = lawofficespoints$id[nearestindices],
  nearest_x = st_coordinates(nearest_points)[, 1],
  nearest_y = st_coordinates(nearest_points)[, 2],
  nearest_distances_mi = nearest_distances_mi,
  geometry = blockgroupcentroids$geometry
)

distanceresults = as.data.frame(distanceresults)
```

```r
nearestdf = distanceresults %>%
  select(nearest_x, nearest_y)

nearestdf = st_as_sf(distanceresults, coords = c("nearest_x", "nearest_y"), crs = 4326)

distanceresults = st_as_sf(distanceresults, crs = 4326)

censuscentroids = st_centroid(censusblocks)
```

```
## Warning: st_centroid assumes attributes are constant over geometries
```

```r
censuscentroids = left_join(censuscentroids, as.data.frame(distanceresults))
```

```
## Joining with `by = join_by(geometry)`
```

```r
max(as.numeric(censuscentroids$nearest_distances_mi))
```

```
## [1] 15.75184
```

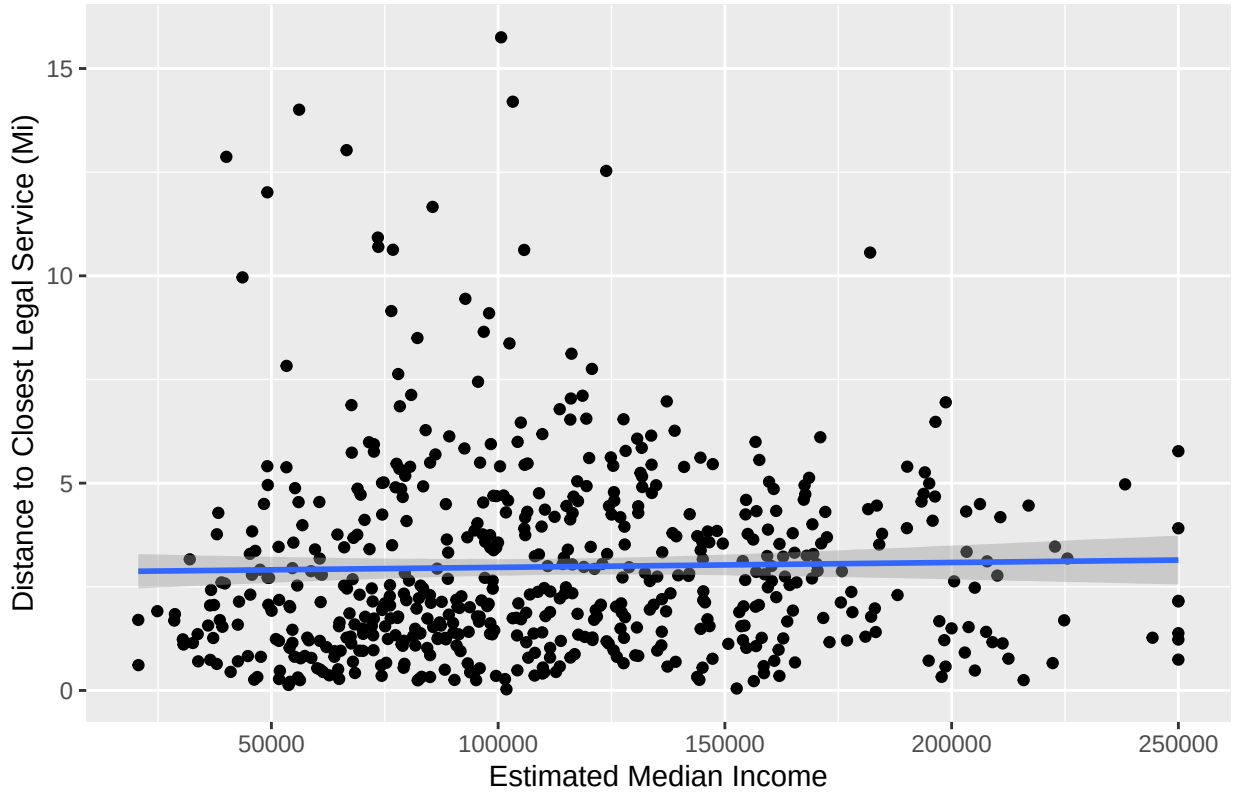
```r
min(as.numeric(censuscentroids$nearest_distances_mi))
```

```
## [1] 0.02776458
```

```r
ggplot(data = censuscentroids, aes(x = estimate, y = as.numeric(nearest_distances_mi)))+
  geom_point() +
  labs(title = "Fig. 5. Distribution of Groups by Income Against Distance from Closest Legal Service",
       x = "Estimated Median Income",
       y = "Distance to Closest Legal Service (Mi)") +
  geom_smooth(method = "lm")
```

```
## `geom_smooth()` using formula = 'y ~ x'

## Warning: Removed 27 rows containing non-finite outside the scale range
## (`stat_smooth()`).

## Warning: Removed 27 rows containing missing values or values outside the scale range
## (`geom_point()`).
```

Fig. 5. Distribution of Groups by Income Against Distance from Closest Lega



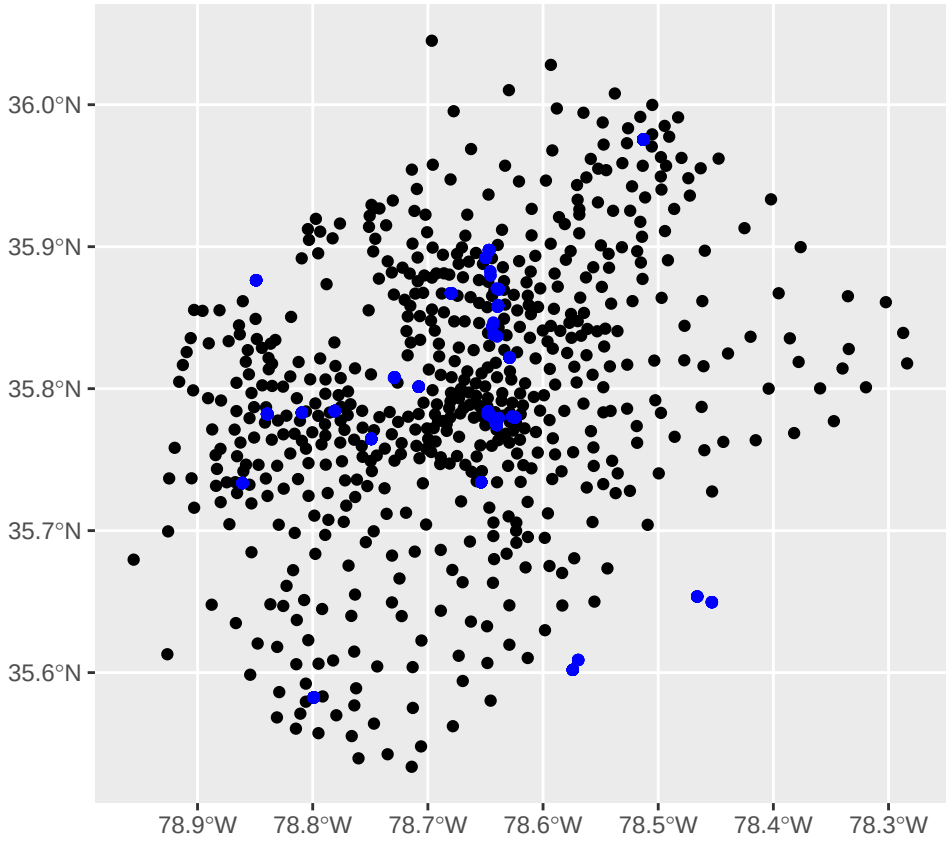

```
fit3 = lm(as.numeric(nearest_distances_mi) ~ estimate, data = censuscentroids)
summary(fit3)
```

```
##
## Call:
## lm(formula = as.numeric(nearest_distances_mi) ~ estimate, data = censuscentroids)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -2.9829 -1.6794 -0.6263  1.2020 12.7829
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) 2.850e+00  2.485e-01  11.471   <2e-16 ***
## estimate    1.178e-06  2.046e-06   0.575    0.565
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 2.363 on 568 degrees of freedom
##   (27 observations deleted due to missingness)
## Multiple R-squared:  0.0005826,  Adjusted R-squared:  -0.001177
## F-statistic: 0.3311 on 1 and 568 DF,  p-value: 0.5652
```

```
distances2 = st_union(distanceresults, nearestdf) %>%  st_cast("LINESTRING")
```

```
## Warning: attribute variables are assumed to be spatially constant throughout
## all geometries
```

```
#This code to turn the two points into a line is taken from stackexchange user Spacedman

ggplot() +
  geom_sf(data = distanceresults, color = "black") +
  geom_sf(data = nearestdf, color = "blue")
```




```
  geom_sf(data = distances2, color = "grey")
```

```
## [[1]]
## mapping:
## geom_sf: na.rm = FALSE
## stat_sf: na.rm = FALSE
## position_identity
##
## [[2]]
## <ggproto object: Class CoordSf, CoordCartesian, Coord, gg>
##     aspect: function
##     backtransform_range: function
##     clip: on
##     crs: NULL
##     datum: crs
##     default: TRUE
```

```
##     default_crs: NULL
##     determine_crs: function
##     distance: function
##     expand: TRUE
##     fixup_graticule_labels: function
##     get_default_crs: function
##     is_free: function
##     is_linear: function
##     label_axes: list
##     label_graticule:
##     labels: function
##     limits: list
##     lims_method: cross
##     modify_scales: function
##     ndiscr: 100
##     params: list
##     range: function
##     record_bbox: function
##     render_axis_h: function
##     render_axis_v: function
##     render_bg: function
##     render_fg: function
##     setup_data: function
##     setup_layout: function
##     setup_panel_guides: function
##     setup_panel_params: function
##     setup_params: function
##     train_panel_guides: function
##     transform: function
##     super:  <ggproto object: Class CoordSf, CoordCartesian, Coord, gg>
```